di.unito.it

DIPARTIMENTO
DI INFORMATICA

TLN

basic text processing

Daniele Radicioni

credits

- this lesson was prepared by also using materials taken from:
  - https://www.nltk.org/book/
  - https://spacy.io/
  - https://realpython.com/nltk-nlp-python

outline

- introduction of some basic NLP tasks:
  - tokenization, stop words filtering, stemming, part-of-speech tagging (POS), lemmatization, Named Entity Recognition (NER)
- dealing with the tasks through the Natural Language Tool Kit (NLTK) library
- dealing with same tasks through spaCy

NLTK

Natural Language Tool-Kit: NLTK

- provides many text processing packages and a large number of test datasets
- many NLP operations, including tokenizing and visualizing parse trees, may be easily accomplished
- this link should be bookmarked:
  - https://www.nltk.org/book/

tokenization

tokenization

- tokenizing a string means splitting it up by word or by sentence.
  - in both cases we obtain smaller pieces of text
- word tokenization: amounts to identifying words, the smallest unit of meaning
- sentence tokenization: a text is split into the sentences it contains

```
from nltk.tokenize import sent_tokenize, word_tokenize


example_string = "IT was 2 p.m. on the afternoon of May 7, 1915. The
Lusitania had been struck and was sinking rapidly, while the boats
were being launched with all possible speed. The women and children
were being lined up awaiting their turn. Some still clung desperately
to husbands and fathers. One girl stood alone, slightly apart from the
rest. She was quite young, not more than eighteen. She did not seem
afraid, and her grave, steadfast eyes looked straight ahead."


sent_tokenize(example_string)
```

```
['IT was 2 p.m. on the afternoon of May 7, 1915.',
 'The Lusitania had been struck and was sinking rapidly, while the boats were
being launched with all possible speed.',
 'The women and children were being lined up awaiting their turn.',
 'Some still clung desperately to husbands and fathers.',
 'One girl stood alone, slightly apart from the rest.',
 'She was quite young, not more than eighteen.',
 'She did not seem afraid, and her grave, steadfast eyes looked straight
ahead.']
```

```
from nltk.tokenize import sent_tokenize, word_tokenize


example_string = "IT was 2 p.m. on the afternoon of May 7, 1915. The
Lusitania had been struck and was sinking rapidly, while the boats
were being launched with all possible speed. The women and children
were being lined up awaiting their turn. Some still clung desperately
to husbands and fathers. One girl stood alone, slightly apart from the
rest. She was quite young, not more than eighteen. She did not seem
afraid, and her grave, steadfast eyes looked straight ahead."


word_tokenize(example_string)
```

```
['IT',
 'was',
 '2',
 'p.m.',
 ... ,
 'eyes',
 'looked',
 'straight',
 'ahead',
 '.']
```

stop-words filtering

```
import nltk
from nltk.tokenize import sent_tokenize, word_tokenize
from nltk.corpus import stopwords
nltk.download("stopwords")
nltk.download("punkt")

stopword = stopwords.words('english')
example_string = "The Lusitania had been struck and was sinking rapidly,
while the boats were being launched with all possible speed."

full_text = example_string.lower() # lowercasing
word_tokens = nltk.word_tokenize(full_text)
print (word_tokens[:20])

filtered_tokens = [word for word in word_tokens if word not in stopword]
print (filtered_tokens[:20])
```

stop-words filtering is implemented here

```
import nltk
from nltk.tokenize import sent_tokenize, word_tokenize
from nltk.corpus import stopw
nltk.download("stopwords")
nltk.download("punkt")

stopword = stopwords.words('english')
example_string = "The Lusitania had been struck and was sinking rapidly,
while the boats were being launched with all possible speed."

# full_text = example_string.lower() # not lowercasing any more
word_tokens = nltk.word_tokenize(example_string)
print (word_tokens[:20])

filtered_tokens = [w for w in word_tokens if w.casefold() not in stopword]
print (filtered_tokens[:20])
```

.casefold() is used to ignore whether the letters in word were uppercase or lowercase.
this is done because `stopwords.words('english')` includes only lowercase versions of stop words.

stop-words filtering is implemented here, in 'ignore-case' mode

a note on filtering

- content words convey information about the topics covered in the text; conversely,
- context words carry information on the writing style
  - these allow to retrieve patterns on how authors use context words thereby characterizing their writing style
  - in principle, after having quantified an author's writing style, we can analyze a text written by an unknown author to see in how far it matches a particular writing style: this way, one can try to identify who the author is (or how stylistically close two authors are)

- listing stopwords for all available languages

```
import nltk
from nltk.corpus import stopwords

nltk.download('stopwords')
print(stopwords.fileids())
```

```
['arabic', 'azerbaijani', 'basque',
'bengali', 'catalan', 'chinese',
'danish', 'dutch', 'english', 'finnish',
'french', 'german', 'greek', 'hebrew',
'hinglish', 'hungarian', 'indonesian',
'italian', 'kazakh', 'nepali',
'norwegian', 'portuguese', 'romanian',
'russian', 'slovene', 'spanish',
'swedish', 'tajik', 'turkish']
```

stemming

stemming

- stemming entails reducing words to their root, which is the core part of a word.
  - e.g., the words "helping" and "helper" share the root "help"
  - it allows you to obtain the basic meaning of a word rather than all the details of how it is being used
  - NLTK has more than one stemmer; the Porter stemmer and Snowball, are by far the more largely used
- understemming and overstemming are two possible errors we can incur in:
  - understemming: two related words should be reduced to the same stem but are not (this is a false negative).
  - overstemming: two unrelated words are wrongly reduced to the same stem (this is a false positive).

```
from nltk.stem import PorterStemmer
from nltk.tokenize import word_tokenize

port_stm = PorterStemmer()

example_string = "The Lusitania had been struck and was sinking rapidly,
while the boats were being launched with all possible speed."

words = word_tokenize(example_string)
stemmed_words = [port_stm.stem(word) for word in words]

print(stemmed_words)
```

```
['the', 'lusitania', 'had', 'been', 'struck', 'and', 'wa', 'sink',
'rapidli', ',', 'while', 'the', 'boat', 'were', 'be', 'launch',
'with', 'all', 'possibl', 'speed', '.']
```

```
from nltk.stem import PorterStemmer
from nltk.tokenize import word_tokenize

# port_stm = PorterStemmer() # let us try another one!
snow_stm = nltk.stem.SnowballStemmer('english')

example_string = "The Lusitania had been struck and was sinking rapidly, while
the boats were being launched with all possible speed."

words = word_tokenize(example_string)
stemmed_words = [snow_stm.stem(word) for word in words]

print(stemmed_words)
```

```
['the', 'lusitania', 'had', 'been', 'struck', 'and', 'was',
'sink', 'rapid', ',', 'while', 'the', 'boat', 'were', 'be',
'launch', 'with', 'all', 'possibl', 'speed', '.']
```

PORTER

```
['the', 'lusitania', 'had',
'been', 'struck', 'and', 'wa',
'sink', 'rapidli', ',', 'while',
'the', 'boat', 'were', 'be',
'launch', 'with', 'all',
'possibl', 'speed', '.']
```

SNOWBALL

```
['the', 'lusitania', 'had',
'been', 'struck', 'and', 'was',
'sink', 'rapid', ',', 'while',
'the', 'boat', 'were', 'be',
'launch', 'with', 'all',
'possibl', 'speed', '.']
```

POS tagging

POS tagging

- part of speech tagging or POS tagging is the task of labeling the words in an input text according to their part of speech
  - DET: determiners are at times included in adjectives, and at times a separated class
  - DET are also known as 'limiting adjectives', and come before nouns and specify something about their quantity, definiteness, or ownership

POS	Role	Examples
Noun	person, place, or thing	sea, roof, Italy
Pronoun	replaces a noun	you, I, she
Determiner		
Adjective	gives information about what a noun is like	efficient, windy, colorful
Verb	action or state of being	learn, is, go
Adverb	provides information on a verb, an adjective, or another adverb	efficiently, always, very
Preposition	gives information about how a noun or pronoun is connected to another word	from, about, at
Conjunction	connects two other words or phrases	so, because, and
Interjection	Is an exclamation	yay, ow, wow!

di.unito.it
Dipartimento di Informatica

types of determiners

- articles: these precede and identify nouns or noun phrases as either specific or nonspecific
- demonstrative determiners (demonstrative adjectives): inform on the placement of a noun in space or time (*this*, *that*, *these*, and *those*)
- distributive determiners are referred to a group or individual parts within a group (each, every, all, and both are distributive determiners)
- interrogative determiners narrow a noun's attributes by asking a question (*whose*, *what*, and *which*)
- possessive determiners (possessive adjectives): the possessive forms of the personal pronouns and can appear before a noun (*my, your, his, her, its, our, their*, and *whose*)
- quantifying determiners and numbers: specify something about the associated nouns by grouping them together or indicating how much or how many of them there are (*many, some, few, any, all*, and *several*)
- relative determiners (relative adjectives), carry information about nouns in noun phrases that introduce relative dependent clauses (*what, whatever, which*, and *whichever*).

```
import nltk
from nltk.tokenize import word_tokenize


nltk.download('averaged_perceptron_tagger')


example_string = "The Lusitania had been struck and was sinking
rapidly, while the boats were being launched with all possible speed."
words = word_tokenize(example_string)


print(nltk.pos_tag(words))
```

POS tagging: a list is being passed as argument to the `pos_tag()` function

```
[('The', 'DT'), ('Lusitania', 'NNP'), ('had', 'VBD'), ('been',
'VBN'), ('struck', 'VBN'), ('and', 'CC'), ('was', 'VBD'),
('sinking', 'VBG'), ('rapidly', 'RB'), (',', ','), ('while',
'IN'), ('the', 'DT'), ('boats', 'NNS'), ('were', 'VBD'), ('being',
'VBG'), ('launched', 'VBN'), ('with', 'IN'), ('all', 'DT'),
('possible', 'JJ'), ('speed', 'NN'), ('.', '.')]
```

what is `pos_tag()` returning?

tagset

```
import nltk
nltk.download('tagsets')

# further information at the URL
# https://www.nltk.org/data.html

nltk.help.upenn_tagset()
```

```
CC: conjunction, coordinating
CD: numeral, cardinal
DT: determiner
IN: preposition or conjunction, subordinating
JJ: adjective or numeral, ordinal
VB: verb, base form
VBD: verb, past tense
VBG: verb, present participle or gerund
VBN: verb, past participle
VBP: verb, present tense, not 3rd person singular
VBZ: verb, present tense, 3rd person singular
```

lemmatization

lemmatization

- lemmatizing reduces words to their canonical form or lemma
  - a lemma is the canonical form, dictionary form, or citation form of a set of word forms
- a lemma is a word that represents a whole group of words, and that group of words is called a lexeme
  - *break*, *breaks*, *broke*, *broken* and *breaking* are forms of the same lexeme, with *break* as the lemma

```
import nltk
from nltk.tokenize import word_tokenize
from nltk.stem import WordNetLemmatizer
nltk.download('wordnet')
nltk.download('punkt')


lemmatizer = WordNetLemmatizer()
example_string = "The Lusitania had been struck and was sinking rapidly,
while the boats were being launched with all possible speed."
words = word_tokenize(example_string)
print(words)
lemmatized_words = [lemmatizer.lemmatize(word) for word in words]
print(lemmatized_words)
```

```
['The', 'Lusitania', 'had', 'been', 'struck', 'and', 'was',
'sinking', 'rapidly', ',', 'while', 'the', 'boats', 'were', 'being',
'launched', 'with', 'all', 'possible', 'speed', '.']
['The', 'Lusitania', 'had', 'been', 'struck', 'and', 'wa',
'sinking', 'rapidly', ',', 'while', 'the', 'boat', 'were', 'being',
'launched', 'with', 'all', 'possible', 'speed', '.']
```

```
import nltk
from nltk.tokenize import word_tokenize
from nltk.stem import WordNetLemmatizer
nltk.download('wordnet')
nltk.download('punkt')


lemmatizer = WordNetLemmatizer()
example_string = "The Lusitania had been struck and was sinking rapidly,
while the boats were being launched with all possible speed."
words = word_tokenize(example_string)
print(words)
lemmatized_words = [lemmatizer.lemmatize(word) for word in words]
print(lemmatized_words)
```

```
['The', 'Lusitania', 'had', 'been', 'struck', 'and', 'was',
'sinking', 'rapidly', ',', 'while', 'the', 'boats', 'were', 'being',
'launched', 'with', 'all', 'possible', 'speed', '.']
['The', 'Lusitania', 'had', 'been', 'struck', 'and', 'wa',
'sinking', 'rapidly', ',', 'while', 'the', 'boat', 'were', 'being',
'launched', 'with', 'all', 'possible', 'speed', '.']
```

- lemmatizers' performance can be improved by adding POS tagging information

```
...
example_string = "The Lusitania had been struck and was sinking rapidly, while the
boats were being launched with all possible speed."
words = word_tokenize(example_string)
lemmatizer = WordNetLemmatizer()
tagged_token_tup = nltk.pos_tag(words) # POS tagging step: produces a list of tuples
print(tagged_token_tup)
for tup in tagged_token_tup:
  token = tup[0]
  pos = tup[1]
  if pos.startswith('J'):
    wn_pos_tag = 'a'
  elif pos.startswith('N'):
    wn_pos_tag = 'n'
  elif pos.startswith('R'):
    wn_pos_tag = 'r'
  elif pos.startswith('V') or pos.startswith('M'):
    wn_pos_tag = 'v'
  else:
    continue
  lemma = lemmatizer.lemmatize(token,pos=wn_pos_tag)
  print(f'input: {token}; POS: {wn_pos_tag}; lemma: {lemma}')
```

```
...
example_string = "The Lusitania had been struck and was sinking rapidly, while the
boats were being launched with all possible speed."
words = word_tokenize(example_stri
lemmatizer = WordNetLemmatizer()
tagged_token_tup = nltk.pos_tag(words) # POS tagging step: produces a list of tuples
print(tagged_token_tup)
for tup in tagged_token_tup:
  token = tup[0]
  pos = tup[1]
  if pos.startswith('J'):
    wn_pos_tag = 'a'
  elif pos.startswith('N'):
    wn_pos_tag = 'n'
  elif pos.startswith('R'):
    wn_pos_tag = 'r'
  elif pos.startswith('V') or pos.startswith('M'):
    wn_pos_tag = 'v'
  else:
    continue
  lemma = lemmatizer.lemmatize(token,pos=wn_pos_tag)
  print(f'input: {token}; POS: {wn_pos_tag}; lemma: {lemma}')
```

WordNet lemmatizer is employed

the second step is POS tagging

loop to convert the POS tag into the naming convention adopted by the WordNet lemmatizer, that expects to have in input 'a', 'n', 'r' or 'v'

call to the lemmatizer, which is now invoked also with the POS tag information

nltk.help.upenn_tagset()

```
# JJ a
# JJR a
# JJS a
# NN n
# NNP n
# NNPS n
# NNS n
# RB r
# RBR r
# RBS r
# RP r
# MD v
# VB v
# VBD v
# VBG v
# VBN v
# VBP v
# VBZ v
```

```
...
example_string = "The Lusitania had been struck and was sinking rapidly, while the
boats were being launched with all possible speed."
words = word_tokenize(example_string)
lemmatizer = WordNetLemmatizer()
tagged_token_tup = nltk.pos_tag(words) # POS tagging step: produces a list of tuples
print(tagged_token_tup)
for tup in tagged_token_tup:
  token = tup[0]
  pos = tup[1]
  if pos.startswith('J'):
    wn_pos_tag = 'a'
  elif pos.startswith('N'):
    wn_pos_tag = 'n'
  elif pos.startswith('R'):
    wn_pos_tag = 'r'
  elif pos.startswith('V') or pos.startswith('M'):
    wn_pos_tag = 'v'
  else:
    continue
  lemma = lemmatizer.lemmatize(token,pos=wn_pos_tag)
  print(f'input: {token}; POS: {wn_pos_tag}; lemma: {lemma}')
```

```
[('The', 'DT'), ('Lusitania', 'NNP'),
('had', 'VBD'), ('been', 'VBN'),
('struck', 'VBN'), ('and', 'CC'), ('was',
'VBD'), ('sinking', 'VBG'), ('rapidly',
'RB'), (',', ',') ...
```

lemmatizer is called by feeding it with token and the wn_pos_tag.

```
input: Lusitania; POS: n; lemma:
Lusitania
input: had; POS: v; lemma: have
input: been; POS: v; lemma: be
input: struck; POS: v; lemma: strike
input: was; POS: v; lemma: be
input: sinking; POS: v; lemma: sink
input: rapidly; POS: r; lemma: rapidly
input: boats; POS: n; lemma: boat
input: were; POS: v; lemma: be
input: being; POS: v; lemma: be
...
```

NER

NER

- named entities are noun phrases that refer to specific locations, people, organizations, and so on.
- the task of named entity recognition (NER) amounts to finding the named entities in text documents, and to determine the kind of named entity

a list of entities

NE type	Examples
ORGANIZATION	Georgia-Pacific Corp., WHO
PERSON	Eddy Bonte, President Obama
LOCATION	Murray River, Mount Everest
DATE	June, 2008-06-29
TIME	two fifty a m, 1:30 p.m.
MONEY	175 million Canadian dollars, GBP 10.40
PERCENT	twenty pct, 18.75 %
FACILITY	Washington Monument, Stonehenge
GPE	South East Asia, Midlothian

```
import nltk

nltk.download("maxent_ne_chunker")
nltk.download("words")

words = word_tokenize("While Ann was coming
home, Jess went to Spain with Joy and her new
Apple laptop. ", language='english')
pos_tags = nltk.pos_tag(words)
tree = nltk.ne_chunk(pos_tags, binary=True)
print(tree)
```

```
(S
  While/IN
  (NE Ann/NNP)
  was/VBD
  coming/VBG
  home/RB
  ,/,
  (NE Jess/NNP)
  went/VBD
  to/TO
  (NE Spain/NNP)
  with/IN
  (NE Joy/NNP)
  and/CC
  her/PRP$
  new/JJ
  Apple/NNP
  laptop/NN
  ./.)
```

```
import nltk

nltk.download("maxent_ne_chunker")
nltk.download("words")

words = word_tokenize("While Ann was coming
home, Jess went to Spain with Joy and her new
Apple laptop. ", language='english')
pos_tags = nltk.pos_tag(words)
tree = nltk.ne_chunk(pos_tags, binary=False)
print(tree)
```

```
(S
  While/IN
  (PERSON Ann/NNP)
  was/VBD
  coming/VBG
  home/RB
  ,/,
  (PERSON Jess/NNP)
  went/VBD
  to/TO
  (GPE Spain/NNP)
  with/IN
  (PERSON Joy/NNP)
  and/CC
  his/PRP$
  new/JJ
  Apple/NNP
  laptop/NN
  ./.)
```

```
import nltk
nltk.download("maxent_ne_chunker")
nltk.download("words")


example_string = "The Lusitania had been struck and was sinking rapidly;
Jenny runaway and started swimming towards the US coast."


def extract_ne(example_string):
    words = word_tokenize(example_string, language='english')
    tags = nltk.pos_tag(words)
    tree = nltk.ne_chunk(tags, binary=True)
    return set(
        " ".join(i[0] for i in t)
        for t in tree
        if hasattr(t, "label") and t.label() == "NE"
    )
extract_ne(example_string)
```

```
{'Jenny', 'Lusitania', 'US'}
```

exploring spaCy

base functionalities

- spaCy is a free, open-source library for Natural Language Processing (NLP) in Python
  - https://spacy.io/
- main functionalities:
  - Tokenization
  - Part-of-speech (POS) Tagging
  - Dependency Parsing
  - Lemmatization
  - Sentence Boundary Detection (SBD)
  - Named Entity Recognition (NER)

advanced functionalities

- more advanced functionalities:
  - Entity Linking (EL): Disambiguating textual entities to unique identifiers in a knowledge base.
  - Similarity: Comparing words, text spans and documents and how similar they are to each other.
  - Text Classification: Assigning categories or labels to a whole document, or parts of a document.
  - Rule-based Matching: Finding sequences of tokens based on their texts and linguistic annotations, similar to regular expressions.
  - Training: Updating and improving a statistical model's predictions.
  - Serialization: Saving objects to files or byte strings.

model

- some of spaCy's features work independently, while other features require trained pipelines to be loaded, which enable spaCy to predict linguistic annotations
  - e.g., to decide whether a word is a verb or a noun
  - a trained pipeline can consist of multiple components that use a statistical model trained on labeled data
  - many languages covered, which we can install as individual Python modules

linguistic annotations

- linguistic annotations are used to give us insights into the grammatical and syntactic structure of text documents
  - this includes word types (such as part of speech), and how the words are syntactically related to each other
  - e.g., when analyzing text, it makes a huge difference whether a noun is the subject of a sentence, or the object – or whether "can" is used as a verb, or refers to a beverage container

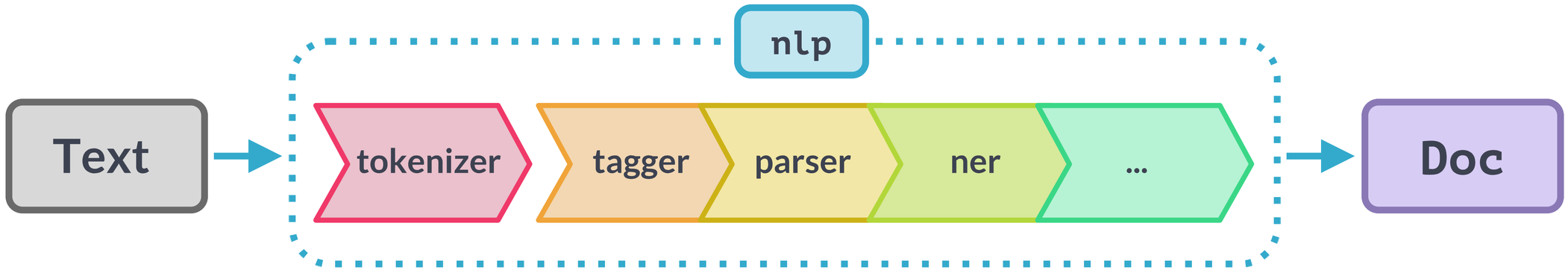
```
!pip install -U spacy
!python -m spacy download en_core_web_sm

import spacy

# spacy.prefer_gpu()
nlp = spacy.load("en_core_web_sm")

doc = nlp("The Lusitania had been struck and
was sinking rapidly; Jenny runaway and started
swimming towards the US coast.")
for token in doc:
    print(token.text, token.pos_, token.dep_)
```

```
The DET det
Lusitania PROPN nsubjpass
had AUX aux
been AUX auxpass
struck VERB ccomp
and CCONJ cc
was AUX aux
sinking VERB conj
rapidly ADV advmod
; PUNCT punct
Jenny PROPN nsubj
runaway ADV ROOT
and CCONJ cc
started VERB conj
swimming VERB xcomp
towards ADP prep
the DET det
US PROPN compound
coast NOUN pobj
. PUNCT punct
```

the pipeline

```
doc = nlp("Jenny ran away and started swimming towards the US coast.")
```

- by calling `nlp` on a text, spaCy first tokenizes the text and produces as output a `Doc` object
  - the `Doc` is then processed in several steps in the pipeline;
  - the processing pipeline typically include a tagger, a lemmatizer, a parser and an entity recognizer; each pipeline component returns the processed `Doc`, which is then passed on to the next component.



getting information on the pipeline

```
nlp = spacy.load("en_core_web_sm")


print(nlp.pipe_names)
print(nlp.pipeline)
```

```
['tok2vec', 'tagger', 'parser', 'attribute_ruler', 'lemmatizer', 'ner']
[('tok2vec', <spacy.pipeline.tok2vec.Tok2Vec object at
0x7da6e8c4d060>), ('tagger', <spacy.pipeline.tagger.Tagger object at
0x7da6e8c4d900>), ('parser',
<spacy.pipeline.dep_parser.DependencyParser object at 0x7da6e8bffa00>),
('attribute_ruler', <spacy.pipeline.attributeruler.AttributeRuler
object at 0x7da6e99277c0>), ('lemmatizer',
<spacy.lang.en.lemmatizer.EnglishLemmatizer object at 0x7da6e98c7ac0>),
('ner', <spacy.pipeline.ner.EntityRecognizer object at
0x7da6e8bff8b0>)]
```

```
texts = ["This is a text", "This is another one", "Last text in this list"]
docs = list(nlp.pipe(texts))

for doc in docs:
  for token in doc:
    print(token.text, token.lemma_, token.pos_)
```

- when processing large volumes of text, the models are more efficient if fed with batches of texts.
  - spaCy's `nlp.pipe` method takes an iterable of texts and outputs processed `Doc` objects, that can be accessed as we formerly did.

```
for docs in nlp.pipe(texts):
   print([(token.lemma_, token.pos_) for token in docs])
```

processing pipeline

- the full pipeline is described at https://spacy.io/api
- if a specific component of the pipeline is not needed –for example, the entity tagger or the parser–, you can disable or exclude it to improve speed. there are two different mechanisms:
  - **`Disable`**: the component and its data will be loaded with the pipeline, but it will be disabled by default and not run as part of the processing pipeline. when you save out the `nlp` object, the disabled component will be included but disabled by default.
  - **`Exclude`**: don't load the component and its data with the pipeline. once the pipeline is loaded, there will be no reference to the excluded component.

```
disabled = nlp.select_pipes(disable="ner")
doc = nlp("I won't have named entities")
[print(token.text, token.lemma_) for token in doc]
disabled.restore()
```

STRING NAME	COMPONENT	DESCRIPTION
tagger	Tagger	Assign part-of-speech-tags.
parser	DependencyParser	Assign dependency labels.
ner	EntityRecognizer	Assign named entities.
entity_linker	EntityLinker	Assign knowledge base IDs to named entities. Should be added after the entity recognizer.
entity_ruler	EntityRuler	Assign named entities based on pattern rules and dictionaries.
textcat	TextCategorizer	Assign text categories: exactly one category is predicted per document.
textcat_multilabel	MultiLabel_TextCategorizer	Assign text categories in a multi-label setting: zero, one or more labels per document.
lemmatizer	Lemmatizer	Assign base forms to words using rules and lookups.
trainable_lemmatizer	EditTreeLemmatizer	Assign base forms to words.
morphologizer	Morphologizer	Assign morphological features and coarse-grained POS tags.
attribute_ruler	AttributeRuler	Assign token attribute mappings and rule-based exceptions.
senter	SentenceRecognizer	Assign sentence boundaries.
sentencizer	Sentencizer	Add rule-based sentence segmentation without the dependency parse.
tok2vec	Tok2Vec	Assign token-to-vector embeddings.
transformer	Transformer	Assign the tokens and outputs of a transformer model.

```
import spacy

# The source pipeline with different components
source_nlp = spacy.load("en_core_web_sm")
print(source_nlp.pipe_names)

# Add only the entity recognizer to the new blank pipeline
nlp = spacy.blank("en")
nlp.add_pipe("ner", source=source_nlp)
print(nlp.pipe_names)
```

```
['tok2vec', 'tagger', 'parser', 'attribute_ruler',
'lemmatizer', 'ner']
['ner']
```

pipeline components

a pipeline component is a function that receives a `Doc` object, modifies it and returns it.

components allow accessing the `Doc` at any point during processing, instead of only being able to modify it afterwards.

```
import spacy
from spacy.language import Language

@Language.component("info_component")
def my_component(doc):
    print(f"After tokenization, this doc has {len(doc)} tokens.")
    print("The POS tags are:", [token.pos_ for token in doc])
    return doc

nlp = spacy.load("en_core_web_sm")
# the component is added at the end of the pipeline
nlp.add_pipe("info_component", name="print_info", last=True)
print(nlp.pipe_names)  # the list contains 'print_info' as the last element
doc = nlp("This is a sentence.") # this produces a call to the new component, too
```

pipeline components

```
import spacy
from spacy.language import Language
@Language.component("info_component")
def my_component(doc):
    print(f"After tokenization, this doc has {len(doc)} tokens.")
    print("The POS tags are:", [token.pos_ for token in doc])
    return doc

nlp = spacy.load("en_core_web_sm")
# the component is added at the end of the pipeline
nlp.add_pipe("info_component", name="print_info", last=True)
print(nlp.pipe_names)              # the list contains 'print_info' as the last element
doc = nlp("This is a sentence.") # this produces a call to the new component, too
nlp.remove_pipe("print_info")     # removes the component from the pipeline
doc = nlp("This is a sentence.") # nothing is printed after this call, since we
                                  # removed the component from the pipeline
```

pipeline components

```
['tok2vec', 'tagger', 'parser', 'attribute_ruler', 'lemmatizer',
'ner', 'print_info']
After tokenization, this doc has 5 tokens.
The POS tags are: ['PRON', 'AUX', 'DET', 'NOUN', 'PUNCT']
```

```
import spacy
from spacy.language import Language
@Language.component("info_component")
def my_component(doc):
    print(f"After tokenization, this doc has {len(doc)} tokens.")
    print("The POS tags are:", [token.pos_ for token in doc])
    return doc

nlp = spacy.load("en_core_web_sm")
# the component is added at the end of the pipeline
nlp.add_pipe("info_component", name="print_info", last=True)
print(nlp.pipe_names)                # the list contains 'print_info' as the last element
doc = nlp("This is a sentence.") # this produces a call to the new component, too
nlp.remove_pipe("print_info")     # removes the component from the pipeline
doc = nlp("This is a sentence.") # nothing is printed after this call, since we
                                 # removed the component from the pipeline
```

using spaCy

repetita iuvant

- as mentioned, processing text with the `nlp` object returns a `Doc` object containing information on tokens, their linguistic features and relationships
  - the returned `Doc` object may be accessed through its fields (such as `.text`)
  - the `Doc` object may be also accessed through spans (slicing indexes are used in this case)

```
doc = nlp("A very simple text example")
print([token.text for token in doc])
```

```
['A', 'very', 'simple', 'text', 'example']
```

```
span = doc[2:4] # index 4 is excluded
print(span)
```

```
simple text
```

```
doc = nlp("A very simple text example.")
print([token.pos_ for token in doc]) # Coarse-grained part-of-speech tags
print([token.tag_ for token in doc]) # Fine-grained part-of-speech tags
```

```
['DET', 'VERB', 'DET', 'NOUN', 'PUNCT'
['DT', 'VBZ', 'DT', 'NN', '.']
```

POS tags

```
print([token.dep_ for token in doc]) # Dependency labels
print([token.head.text for token in doc]) # Syntactic head token
```

```
['det', 'advmod', 'amod', 'compound', 'ROOT', 'punct']
['example', 'simple', 'example', 'example', 'example', 'example']
```

dep and heads

```
doc = nlp("John Lennon was born in Liverpool")
print([(ent.text, ent.label_) for ent in doc.ents]) # Text and label of
named entity span
```

```
[('John Lennon', 'PERSON'), ('Liverpool', 'GPE')]
```

ents

```
doc = nlp("This the first sentence. This is the second one.")
print([sent.text for sent in doc.sents])
```

```
['This the first sentence.', 'This is the second one.']
```

```
doc = nlp("The Lusitania had been struck and was sinking rapidly; Jenny
runaway and started swimming towards the US coast.")
print([chunk.text for chunk in doc.noun_chunks])
```

```
['The Lusitania', 'Jenny', 'the US coast']
```

```
print(spacy.explain("RB"))  # 'adverb'
print(spacy.explain("GPE")) # 'Countries, cities, states'
```

```
adverb
Countries, cities, states
```

to use vectors larger models (ending in md or lg) must be downloaded and employed

```
!python -m spacy download en_core_web_lg
import en_core_web_lg
nlp = en_core_web_lg.load()

import en_core_web_lg
nlp = en_core_web_lg.load()

doc1 = nlp("I like cats")
print(doc1[2].vector)      # Vector as a numpy array
print(doc1[2].vector_norm) # The L2 norm of the vector, tells the vector's magnitude

doc2 = nlp("I like dogs")

print(f'doc1.similarity(doc2): {doc1.similarity(doc2)}')                  # Compare 2 documents
print(f'doc1[2].similarity(doc2[2]): {doc1[2].similarity(doc2[2])}')      # Compare 2 tokens
print(f'doc1[0].similarity(doc2[1:3]): {doc1[0].similarity(doc2[1:3])}') # Compare token and span
```

filtering stop-words

```
import spacy
spacy_stopwords = spacy.lang.en.stop_words.STOP_WORDS
print(f'overall {len(spacy_stopwords)} stopwords are being employed...')

# for stop_word in list(spacy_stopwords)[:10]:
#   print(stop_word)

input_text = "The Lusitania had been struck and was sinking rapidly; Jenny
runaway and started swimming towards the US coast."

nlp = spacy.load("en_core_web_sm")
doc = nlp(input_text)
print([token for token in doc if not token.is_stop])
```

```
overall 326 stopwords are being employed...
[Lusitania, struck, sinking, rapidly, ;, Jenny, runaway, started, swimming,
coast, .]
```

lemmatization

```
import spacy
nlp = spacy.load("en_core_web_sm")
input_text = "The Lusitania had been struck and was sinking rapidly; Jenny
runaway and started swimming towards the US coast."


doc = nlp(input_text)
for token in doc:
    # if str(token) != str(token.lemma_):
    print(f"{str(token):>10} : {str(token.lemma_)}")
```

```
       The : the
 Lusitania : Lusitania
       had : have
      been : be
    struck : strike
...
```

word frequency

```
text = "An operating system (OS) is system software that manages computer" \
    "hardware and software resources, and provides common services for" \
    "computer programs. Time-sharing operating systems schedule tasks " \
    "for efficient use of the system and may also include accounting " \
    "software for cost allocation of processor time, mass storage, " \
    "peripherals, and other resources. For hardware functions such as " \
    "input and output and memory allocation, the operating system acts " \
    "as an intermediary between programs and the computer hardware, " \
    "although the application code is usually executed directly by the " \
    "hardware and frequently makes system calls to an OS function or is " \
    "interrupted by it. Operating systems are found on many devices that " \
    "contain a computer – from cellular phones and video game consoles to " \
    "web servers and supercomputers."
doc = nlp(text)
words = [
    token.text
    for token in doc
    if not token.is_stop and not token.is_punct
]
print(Counter(words).most_common(5))
```

```
[('system', 5),
('operating', 3),
('software', 3),
('hardware', 3),
('OS', 2)]
```

POS tagging

```
import spacy
nlp = spacy.load("en_core_web_sm")
input_text = "The Lusitania had been struck and was sinking rapidly."

doc = nlp(input_text)
for token in doc:
  print(
        f'TOKEN: {str(token):10} \
        TAG: {str(token.tag_):10} \
        POS: {str(token.pos_):10} \
        EXPLANATION: {spacy.explain(token.tag_):20}'
       )
```

```
TOKEN: The        TAG: DT    POS: DET     EXPLANATION: determiner
TOKEN: Lusitania  TAG: NNP   POS: PROPN   EXPLANATION: noun, proper singular
TOKEN: had        TAG: VBD   POS: AUX     EXPLANATION: verb, past tense
TOKEN: been       TAG: VBN   POS: AUX     EXPLANATION: verb, past participle
TOKEN: struck     TAG: VBN   POS: VERB    EXPLANATION: verb, past participle
TOKEN: and        TAG: CC    POS: CCONJ   EXPLANATION: conjunction, coordinating
TOKEN: was        TAG: VBD   POS: AUX     EXPLANATION: verb, past tense
TOKEN: sinking    TAG: VBG   POS: VERB    EXPLANATION: verb, gerund or present participle
TOKEN: rapidly    TAG: RB    POS: ADV     EXPLANATION: adverb
TOKEN: .          TAG: .     POS: PUNCT   EXPLANATION: punctuation mark, sentence closer
```

extracting tokens with specific POS

```
proper_nouns = []
verbs = []
for token in doc:
    if token.pos_ == "PROPN":
        proper_nouns.append(token)
    if token.pos_ == "VERB":
        verbs.append(token)
print(proper_nouns)
print(verbs)
```

```
[Lusitania]
[struck, sinking]
```

shallow parsing

extracting NPs

```
import spacy
nlp = spacy.load("en_core_web_sm")

input_text = "The Lusitania had been struck and was sinking rapidly;
Jenny runaway and started swimming towards the US coast."

doc = nlp(input_text)

for chunk in doc.noun_chunks:
    print(chunk)
```

```
The Lusitania
Jenny
the US coast
```

extracting VPs

```
!python -m pip install textacy
import textacy

text = "The Lusitania had been struck and was sinking rapidly; Jenny runaway and
started swimming towards the US coast."

patterns = [{"POS": "AUX"}, {"POS": "VERB"}]
doc = textacy.make_spacy_doc(text, lang="en_core_web_sm")
verb_phrases = textacy.extract.token_matches(doc, patterns=patterns)

for chunk in verb_phrases:
    print(chunk.text)
```

```
been struck
was sinking
```

Named-Entity Recognition

```
import spacy
nlp = spacy.load("en_core_web_sm")

text = "The Lusitania had been struck and was sinking rapidly; Jenny runaway and started swimming towards the
US coast."
doc = nlp(text)

for ent in doc.ents:
  print(f'{ent.text:10} {ent.start_char:10} {ent.end_char:10} {ent.label_:10} {spacy.explain(ent.label_):10}')
```

```
Jenny                55         60 PERSON     People, including fictional
US                  102        104 GPE        Countries, cities, states
```


UNIVERSITAS STUDII TAURINENSIS
1404